

Free-Surface 2D Fluid Solver Report

(Labs 6, 7, 8)

Léonard Mauvernay



Abstract

For the second project of CSC_3S006_EP, I implemented a free-surface 2D fluid solver based on incompressible Euler's equations. Lab 6 builds a Voronoï diagram via Voronoï Parallel Linear Enumeration with Sutherland-Hodgman clipping. Lab 7 extends it to a power diagram and optimizes the weights with semi-discrete optimal transport via L-BFGS. Lab 8 implements the Gallouët-Mérigot scheme with partial optimal transport, disk clipping of each Laguerre cell, and spring force toward each cell centroid. All timings are on a MacBook Air M2.

Setup

The simulation domain is the unit square $[0, 1]^2$. The course template [2] provided the `Vector` and `Polygon` classes, the `save_frame` rasterizer, the `save_svg` writer, the `progress` callback, the L-BFGS wrapper `OptimalTransport::optimize`, and the `Fluid` class shell with `run_simulation`.

I wrote `clip_by_bisector`, `clip_by_edge`, the site loop in `compute`, the three `Polygon` methods, the objective and gradient in `evaluate`, the `Fluid` constructor body, and `time_step`. The solver uses libLBFGS [3]. Random site and particle initialization use `rand()` [4]. I did not integrate nanoflann so diagram construction is the naive $O(N^2)$ version.

Lab 6. Voronoï Parallel Linear Enumeration

Method. I compute each Voronoï cell independently and in parallel. Starting from the unit square, I clip the current cell by the bisector of $[P_0, P_j]$ for every other site P_j . Each clip is one half-space step of the Sutherland-Hodgman algorithm. A vertex X is kept when

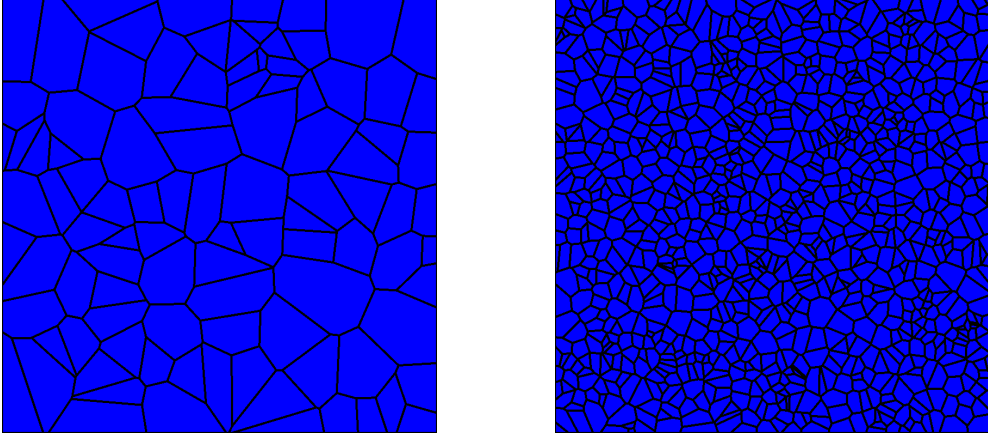
$$\|X - P_0\|^2 \leq \|X - P_j\|^2. \quad (1)$$

When an edge $[A, B]$ crosses the bisector, the intersection point is

$$M = \frac{P_0 + P_j}{2}, \quad t = \frac{\langle M - A, P_j - P_0 \rangle}{\langle B - A, P_j - P_0 \rangle}, \quad P = A + t(B - A). \quad (2)$$

I apply the four Sutherland-Hodgman cases. If B is inside and A is outside, I add P then B . If both are inside, I add B . If B is outside and A is inside, I add P . If both are outside, I add nothing.

Implementation. The clip is in `clip_by_bisector` with weights set to zero. The output polygon reserves capacity for the input polygon's vertex count plus one, as advised by the teacher. The `compute` method loops over all sites with `#pragma omp parallel for`. Figure 1 shows the result.



(a). $N = 100$ random sites, 1.8 ms.

(b). $N = 1000$ random sites, 14.1 ms.

Figure 1. Voronoi diagram of uniform random sites in $[0, 1]^2$

Lab 7. Power diagram and semi-discrete optimal transport

Power diagram. The power cell of site P_0 with weight w_0 is the set of points x satisfying

$$\|x - P_0\|^2 - w_0 \leq \|x - P_j\|^2 - w_j \quad \forall j \neq 0 \quad (3)$$

The Sutherland-Hodgman loop is unchanged. The inside test becomes (3). The midpoint M of (2) is replaced by

$$M' = \frac{P_0 + P_j}{2} + \frac{w_0 - w_j}{2\|P_j - P_0\|^2} (P_j - P_0) \quad (4)$$

Note that when $w_0 = w_j$ this reduces to the bisector. So `clip_by_bisector` handles both labs.

Optimal transport. I optimize the weights by maximizing the semi-discrete optimal transport functional (y_i denotes the i -th site). With uniform density $f = 1$ and target masses λ_i , the functional is

$$g(W) = \sum_i \int_{\text{Pow}_W(y_i)} (\|x - y_i\|^2 - w_i) dx + \sum_i \lambda_i w_i \quad (5)$$

whose gradient is

$$\frac{\partial g}{\partial w_i}(W) = \lambda_i - \text{Area}(\text{Pow}_W(y_i)) \quad (6)$$

I triangulate each cell as a fan from its first vertex. Each triangle T with vertices c_1, c_2, c_3 contributes

$$\int_T \|x - y_i\|^2 dx = \frac{|T|}{6} \sum_{1 \leq k \leq l \leq 3} \langle c_k - y_i, c_l - y_i \rangle \quad (7)$$

Cell area is then computed with

$$\text{Area} = \frac{1}{2} \left| \sum_{k=0}^{n_v-1} (X_k Y_{k+1} - X_{k+1} Y_k) \right| \quad (8)$$

where (X_k, Y_k) are the n_v cell vertices (indices modulo n_v). The triangle integral is in `integral_square_distance` and the area formula is in `area`.

L-BFGS coupling. `libLBFGS` minimizes, so the `evaluate` callback returns $-g$ and the gradient of $-g$. In Lab 7 all targets are equal, so $\lambda_i = \lambda = 1/n$, where n is the number of sites. For each cell I compute the area A_i and the integral from (7).

The per-cell contribution to g is

$$g_i = \int_{\text{Pow}_W(y_i)} \|x - y_i\|^2 dx - w_i A_i + \lambda w_i \quad (9)$$

with w_i stored as `x[i]` in the code. I subtract g_i from `fx` and set `g[i] = A_i - λ`, which is $-\partial g / \partial w_i$ from (6)

Integer-division bug. My first version wrote `lambda = 1/n` with integer operands, giving 0 for any $n > 1$. The gradient then did not drive the cells toward equal areas. The fix is `lambda = 1.0 / n`. I was careful to avoid the same mistake elsewhere in the code.

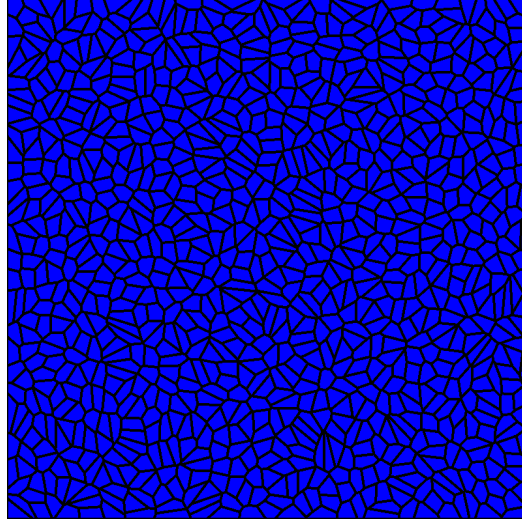


Figure 2. Power diagram of 1000 random sites after L-BFGS optimization with $\lambda = 1/n$

The sites are fixed. Only the weights change to enforce equal cell areas.

Lab 8. Gallouët-Mérigot incompressible Euler scheme

Partial transport. I simulate a fluid occupying fraction $V_f = 0.4$ of the unit square (`fluid_volume` in the code). I keep one weight w_i per fluid particle and one shared air weight w_{air} , so the solver optimizes $N + 1$ variables. The maximized functional is

$$g(W) = \sum_{i=1}^N \int_{\text{Pow}_W(y_i)} (\|x - y_i\|^2 - w_i) dx + \frac{V_f}{N} \sum_{i=1}^N w_i + w_{\text{air}}(s - V_f) \quad (10)$$

where $s = \sum_{i=1}^N \text{Area}(\text{Pow}_i)$ is the total fluid area and Pow_i denotes the fluid cell after disk clipping (what `compute` stores in `cells[i]`). The gradients of g are

$$\frac{\partial g}{\partial w_i} = \frac{V_f}{N} - \text{Area}(\text{Pow}_i), \quad \frac{\partial g}{\partial w_{\text{air}}} = s - V_f \quad (11)$$

The air branch runs when `n > N_fluid`. Since `libLBFGS` minimizes $-g$, the code sets `g[N_fluid] = fluid_volume - s`, which is $-\partial g / \partial w_{\text{air}}$ and subtracts `x[N_fluid] * (s - fluid_volume)` from `fx`. The fluid loop is the Lab 7 code with $\lambda = V_f / N$.

Disk intersection. The air constraint is

$$\|x - y_i\|^2 - w_i \leq -w_{\text{air}}$$

so each fluid power cell is intersected with a disk centered at y_i of radius

$$R_i = \sqrt{w_i - w_{\text{air}}} \quad (12)$$

I approximate the disk with $M = 80$ sides and apply `clip_by_edge` to each side. The edge from u to v uses $N_d = (u_y - v_y, v_x - u_x)$. A vertex X is inside when $\langle X - u, N_d \rangle \geq 0$.

The crossing point is $P = A + t(B - A)$ with

$$t = \frac{\langle u - A, N_d \rangle}{\langle B - A, N_d \rangle} \quad (13)$$

Note that when $w_i - w_{\text{air}} \leq 0$, the disk has no positive area, so I empty the cell.

Time step. Each frame runs one Gallouët-Mérigot step. The optimal transport solve computes the weights and Laguerre cells for the current particle positions. For each particle, C_i is set to the cell centroid when the cell has at least three vertices and to the particle position otherwise. I apply

$$F_i = \frac{1}{\varepsilon^2} (C_i - X_i^t) + m_i \vec{g}, \quad v_i^{t+1} = v_i^t + \frac{dt}{m_i} F_i, \quad X_i^{t+1} = X_i^t + dt v_i^{t+1} \quad (14)$$

with $\varepsilon = 0.004$, $m_i = 200$, $dt = 0.01$, and $\vec{g} = (0, -9.81)$. The centroid is

$$C_i = \frac{1}{6 A_s} \sum_{k=0}^{n_v-1} (V_k + V_{k+1}) (X_k Y_{k+1} - X_{k+1} Y_k), \quad A_s = \frac{1}{2} \sum_{k=0}^{n_v-1} (X_k Y_{k+1} - X_{k+1} Y_k) \quad (15)$$

with $V_k = (X_k, Y_k)$ the n_v cell vertices (indices modulo n_v). The position update in (14) uses the updated velocity v_i^{t+1} . Particles leaving the square are mirrored back and the matching velocity component is flipped.

Initialization. By rejection sampling, I place $N = 1000$ particles in a disk of radius $\sqrt{V_f/\pi}$ centered at $(0.5, 0.5)$. I set every fluid weight to $V_f/(N\pi)$ and the air weight to 0. The initial clipping disk of (12) then has area $\pi \cdot V_f/(N\pi) = V_f/N$, matching the target mass per cell. The solver reuses the previous frame's weights as its starting point.

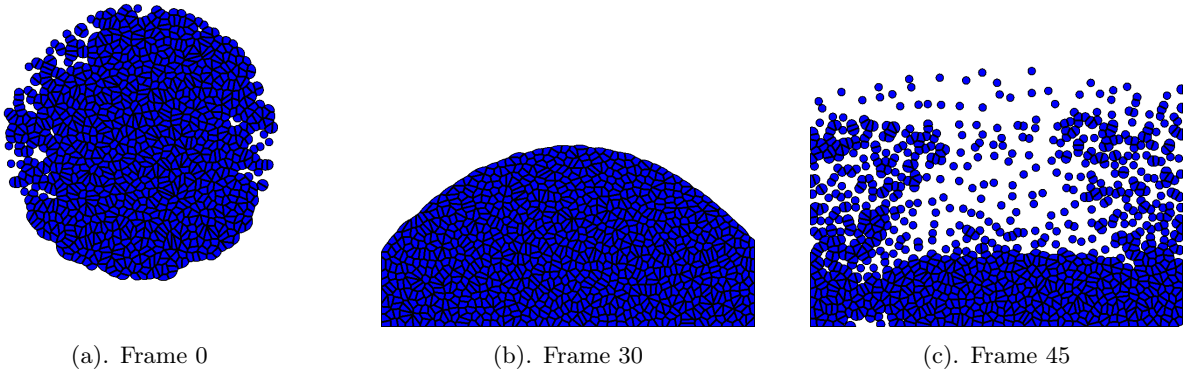


Figure 3. Early phase of the simulation. $N = 1000$ particles, $V_f = 0.4$, $M = 80$ disk sides, $dt = 0.01$, $\varepsilon = 0.004$, $m_i = 200$, $\vec{g} = (0, -9.81)$

Wall collision bug. My original code reflects the velocity on wall contact (`velocities[i][j] *= -1`) as a perfect elastic bounce. Energy builds up over time since no wall collision ever dissipates it. Thus particles never settle. The fix zeros the velocity component on contact and mirrors the position (`particles[i][j] = -particles[i][j]` for the lower wall, `2 - particles[i][j]` for the upper wall). I noticed this bug and fixed it while writing this report. The corrected file `main-lab8-fixed.cpp` is in the repository.

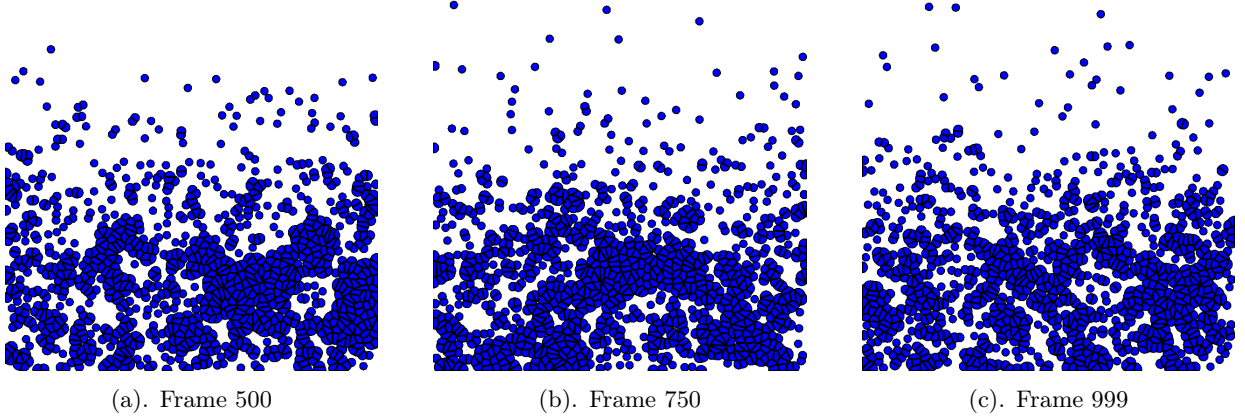


Figure 4. Late phase, unfixed wall collision. Particles keep bouncing and never settle

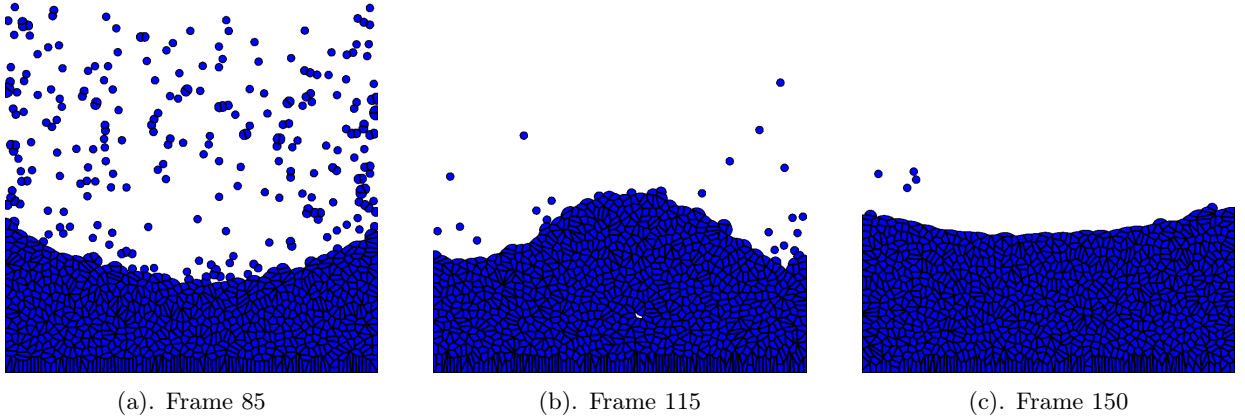


Figure 5. Fixed wall collision. The fluid settles into a flat layer

Each fluid cell is constrained to area V_f/N and so the total fluid area stays close to V_f . Under gravity the fluid falls and spreads along the floor.

Performance

Table 1 collects the measured times (all from the unfixed version). The Voronoï row is one full diagram build. The power-diagram row is the full L-BFGS optimization, which rebuilds the diagram once per solver iteration. The fluid per-frame time is the average `time_step` duration, excluding `save_frame`.

Configuration	Time
Voronoï diagram, $N = 100$	1.8 ms
Voronoï diagram, $N = 1000$	14.1 ms
Power diagram + L-BFGS, 1000 sites, full solve	5.87 s
Fluid <code>time_step</code> , 1000 particles, per frame (unfixed)	3.952 s
Fluid run, 1000 particles, 1000 frames, with saving (unfixed)	3986.85 s

Table 1. Times on a MacBook Air M2, compiled with `-O3` and OpenMP enabled

The diagram build scales with N^2 . Most of the fluid run time is the L-BFGS solve because every iteration rebuilds the full power diagram. The warm start lowers the iteration count on later frames. Moreover, I noticed the iteration count per frame increases once particles reach the floor in the run of the fixed version. it is possibly because the tightly packed configuration makes the optimal transport harder to solve. The fixed run was very slow to complete, so `output_fixed.mp4` covers only the first 350 frames. Both videos are in the repository.

Acknowledgments and external sources

I wrote all the code. As in the first project, I discussed parts of the implementation and ideas with Louis Legrain during the lab sessions. The teacher Nicolas Bonneel and TA Xianjin Gong gave me several advices during the labs (e.g using `reserve()` for the output polygon). Moreover, I used CPreference and Stack Overflow [4] (e.g for random initialization) a few times during my work. An AI assistant (ChatGPT) was used to only generate shell commands such as the OpenMP compilation command.

Course feedback

For me, the incremental lab structure worked well for both projects. The first project had a good pace and I found it really interesting. The second project is harder in my opinion and the step from lab 7 to lab 8 concentrates a lot of new ideas. An extra lab session, or replacing lab 5 with an intermediate fluid step would potentially help. Thank you again for this semester ;)

References

- [1] N. Bonneel, *Computer Graphics, lecture notes (CSC 3S006 EP)*, <https://moodle.ip-paris.fr/mod/resource/view.php?id=181518>.
- [2] N. Bonneel, *fluid_template*, https://github.com/nbonneel/fluid_template.
- [3] N. Okazaki, *libLBFGS, a library of Limited-memory BFGS*, <https://github.com/chokkan/liblbfgs>.
- [4] StackOverflow, *Generate a random double uniformly in $[0, 1]$* , <https://stackoverflow.com/questions/1340729/>.

A Note on labs 5 and 9

The code for lab 5 is in the pathtracer repository and the one of lab 9 is in the `tutte_template` repository.

B Compilation

The command below builds the project on macOS with OpenMP and libLBFGS.

```
clang++ main.cpp lbfgs.c -O3 -march=native -std=c++17 \  
-Xpreprocessor -fopenmp \  
-I"${brew --prefix libomp}/include" \  
-L"${brew --prefix libomp}/lib" \  
-lomp -Wno-deprecated-declarations -o main
```